metaprogramming/trace_calls.rb
```
module TraceCalls
  def self.included(klass)
    klass.instance_methods(false).each do |existing_method|
      wrap(klass, existing_method)
    end

    def klass.method_added(method) # note: nested definition
      unless @trace_calls_internal
        @trace_calls_internal = true
        TraceCalls.wrap(self, method)
        @trace_calls_internal = false
      end
    end
  end

  def self.wrap(klass, method)
    klass.instance_eval do
      method_object = instance_method(method)

      define_method(method) do |*args, &block|
        puts "==> calling #{method} with #{args.inspect}"
        result = method_object.bind_call(self, *args, &block)
        puts "<== #{method} returned #{result.inspect}"
        result
      end
    end
  end
end
```

When we include this module in a class, the included hook method gets invoked. It first uses the instance_methods reflection method to find all the existing instance methods in the host class (the false parameter limits the list to methods in the class itself, and not in its superclasses). For each existing method, the module calls a helper method, wrap, to add some tracing code to it. We'll talk about wrap shortly.

Next, the included method uses another hook, method_added. This is called by Ruby whenever a method is defined in the receiver. Note that we define this method in the class passed to the included method. This means that the method will be called when methods are added to this host class and not to the module. This is what allows us to include TraceCalls at the top of a class and then add methods to that class—all those method definitions will be handled by method_added.

Now look at the code inside the method_added method. We have to deal with a potential problem here. As you'll see when we look at the wrap method, we add tracing to a method by creating a new version of the method that calls the old. Inside method_added, we call the wrap function to add this tracing. But inside wrap, we'll define a new method to handle this wrapping, and that definition will invoke method_added again, and then we'd call wrap again, and so on, until the stack gets exhausted. To prevent this, we use an instance variable and do the wrapping only if we're not already doing it.

The wrap method takes a class object and the name of a method to wrap. It finds the original definition of that method (using instance_method) and saves it. It then redefines this method. This new method outputs some tracing and then calls the original, passing in the parameters